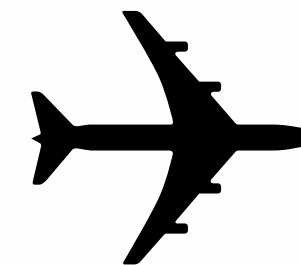


FLIGHT BOOKING SYSTEM



HAYA AL-HASHMI	24-0613
ANFAL AL-SHIZAWI	24-0331
TAIMAA AL-BATRANI	24-0080
RAHAF AL-KALBANI	24-0407



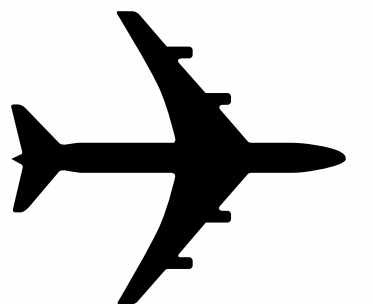
DESCRIPTION OF OUR SYSTEM

Our system is a Flight Booking System made to simplify and manage airline reservation processes easily.

The main purpose of this system is to give users a way to find flights, book them, and cancel them if they need to and keep track of their reservations.

Administrators can manage gates and have control over all the flights and bookings.

Admins will also be able to add and cancel flights and look up flight data of who is on what flight.



MAIN CLASSES

- **Booking:** Users book flights through this class. Admins can also manage and cancel bookings.
- **Flight:** Holds available flights created by admins and contain information like destination, date, time, and seat options..
- **User:** Used to create usernames and personal information. Can check available flights for booking.
- **Guest:** used by guests who want to book available flights. They have less behaviors than those who have admin access.
- **Admin:** Used by staff of the airport to create flights and manage guest bookings. They also have control over opening and closing Gates.

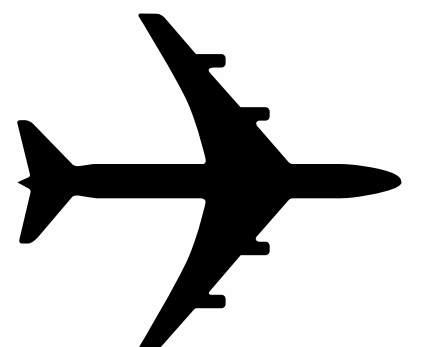
ATTRIBUTES & BEHAVIORS

- **BOOKING:**

- Attributes: destination, date, time, seatNo, gateNo, flightNo.
- Behavior: passengerInfo(); / cancelBooking(); / bookingInfo();

- **FLIGHT :**

- Attributes: departure, arrival, destination, date, flight_num, gate_number, seatClass, vector booking, and static int countBooking.
- Behavior: display_flight();/ bookFlight();/ generateSeat();/ isFull();/ getters/setters, operator==, and showInfo();



ATTRIBUTES & BEHAVIORS

- **USER:**

- Attributes: username, phone number.
- Behavior: *Accesses *bookingInfo()* and *cancelBooking()* from BOOKING.

- **GUEST:**

- Attributes: guestUsername.
- Behavior: chooseSeat() / checkAvailable() / *Accesses *bookingInfo()*, *bookFlight()* and *cancelBooking()* from BOOKING.

- **ADMIN:**

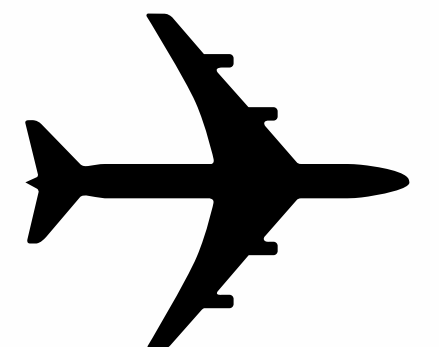
- Attributes: adminUsername.
- Behavior: addFlight() / *Accesses *bookingInfo()*, and *cancelBooking()* from BOOKING.

INHERITANCE AND COMPOSITION

Admin and Guest should inherit from User because Guest and Admin have a IS-A relationship with User

Booking has the concept of composition with User because they have a HAS-A relationship (Passenger HAS-A name and phone number).

Booking objects are fully managed by the Flight class and are stored in a vector. When a Flight object is destroyed, all associated Booking objects are also destroyed, demonstrating composition.

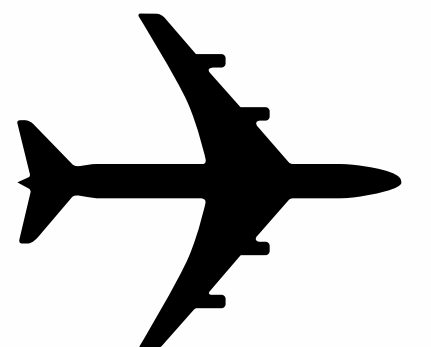


USER-ADMIN-GUEST

Guest and Admin inherit from the abstract class User.

User contains all getters and setters for Admin and Guest.

User also contains a pure virtual function login() that is defined differently in Admin and Guest.



USER-ADMIN-GUEST

Admin contains the function createFlight() that takes a Flight object and can edit it's Departure from, Arrival Time, etc.

```
void createFlight(Flight &f, string departure_from, string arrival_dest,  
int flight_num, string destination ){  
    f.setDeparture(departure_from);  
    f.setArrival(arrival_dest);  
    f.setFlightNum(flight_num);  
    f.setDestination(destination);  
}
```


CLASS BOOKING

Class Booking is responsible for handling all the booking information related to a specific passenger. This works due to the concept of composition because *Booking* contains a pointer to *User** .

Additionally, we have a initializer-list based constructor that initializes passenger details when a *Booking b* object is created.

Lastly, a destructor was added to be invoked automatically when an object is destroyed.

BOOKING.H

```
12  class Booking:
13  protected:
14      User *user;//composition between user and booking
15      string destinationFrom, destinationTo, date, time, gateNo, seatClass, flight;
16      int seatNo;
17      bool isCancelled;
18
19  public:
20      Booking(User* u , string destinationFrom, string destinationTo, string date, string time, string gateNo, string seatClass,
21      string flight, int seatNo) : user(u), destinationFrom(destinationFrom), destinationTo(destinationTo), date(date),
22      time(time), gateNo(gateNo), seatClass(seatClass), flight(flight), seatNo(seatNo), isCancelled(false){}
23      //pssngrInfo is a friend of user (to get his info)
24      void passengerInfo();
25      void cancelBooking();
26      void bookingInfo();
27
28      ~Booking(){}
29  };
30
```

BOOKING.CPP

```
68 void Booking::passengerInfo(){
69     cout << "Full Name: " << user->getUser_name() << endl;
70     cout << "Phone Number: " << user->getPhone_no() << endl;
71 }
72 void Booking::cancelBooking(){
73     isCancelled = true;
74     cout << "Booking cancelled.\n";
75 }
76 void Booking::bookingInfo(){
77     cout << "\n----- Booking Info ----- \n";
78
79     if (isCancelled){
80         cout << "Status: Cancelled\n";
81     } else {
82         cout << "Status: Confirmed\n";
83         cout << "Passenger Name: " << user->getUser_name() << endl;
84         cout << "Date: " << date << endl;
85         cout << "Flight: " << flight << endl;
86         cout << endl;
87         cout << "Boarding Time: " << time << endl;
88         cout << endl;
89         cout << "Destination from " << destinationFrom << " to " << destinationTo << endl;
90         cout << "class: " << seatClass << endl;
91         cout << "Seat Number: " << seatNo << endl;
92         cout << "Gate Number: " << gateNo << endl;
93     }
94 }
95
```

- *passengerInfo()* displays the Passenger's name and phone number from User class.
- *isCancelled()* is responsible of changing booking status to cancelled.
- *bookingInfo()* displays full booking details IFF the booking was confirmed, otherwise it prints "Status: Cancelled".

CLASS FLIGHT

Class Flight is responsible for managing flight details and for handling passenger bookings for each flight.

Composition is applied by storing multiple Booking objects in a vector. The class includes constructors for initialization, and key functions like bookFlight() and generateSeat() to manage reservations.

Operator overloading (operator==) is used to compare flights by flight number, and a static variable countBooking tracks total bookings.

```

class Flight {
private:
    string departure;
    string arrival;
    int flight_num;
    string destination;
    string seatClass;
    string date;
    int gate_number;
    vector<Booking> booking; //flight contains multiple bookings
public:
    Flight(string dep, string arr, int flnum, string des, string d, int gate );    //constructor
    bool operator==(const Flight& other);    // operator == overloading
    static int countBooking; // static counter
    Flight();
    void display_flight();
    void setDeparture(string dep); //setters and getters
    string getDeparture(); //setters and getters
    void setArrival(string arr); //setters and getters
    string getArrival(); //setters and getters
    void setFlightNum(int num); //setters and getters
    int getFlightNum(); //setters and getters
    void setDestination(string des); //setters and getters
    string getDestination(); //setters and getters
    bool isFull();    //after 20 bookings you won't be able to book ( flight is full )
    int generateSeat(string seatClass);    //seat randomizer
    void bookFlight(User* currentUser);    // deciding the seat class
    string showInfo();    //showing how many bookings per flight
    friend class Admin;
};

```

FLIGHT.H

FLIGHT.CPP

```
#include <iostream>
#include <string>
#include<vector>
#include<cstdlib>
#include<ctime>
using namespace std;
#include "Flight.h"
#include "Booking.h"

Flight::Flight(string dep, string arr, int flnum, string des, string d, int gate ){
    departure = dep;
    arrival = arr;
    flight_num = flnum;
    destination = des;
    date = d;
    gate_number = gate;
    srand(time(0));
}

Flight::Flight(){
    srand(time(0)); // initializes random once } // default constructor

bool Flight::operator==(const Flight& other){
    return this->flight_num == other.flight_num; } // operator == overloading

int Flight::countBooking = 0; // static variable initialization

void Flight::display_flight(){
    cout << "Flight number " << flight_num << endl;
    cout << "Departure time " << departure << endl;
    cout << "Arrival time " << arrival << endl;
    cout << "Destination " << destination << endl;
    cout << "Date " << date << endl;
    cout << "Gate " << gate_number << endl; }
```

```
void Flight::setDeparture(string dep){ departure = dep; }
string Flight::getDeparture(){ return departure; }

void Flight::setArrival(string arr){ arrival = arr; }
string Flight::getArrival(){ return arrival; }

void Flight::setFlightNum(int num){ flight_num = num; }
int Flight::getFlightNum(){ return flight_num; }

void Flight::setDestination(string des){ destination = des; }
string Flight::getDestination(){ return destination; }

bool Flight::isFull(){
    return booking.size() >= 20; // max seats } // after 20 bookings you wont be able to book ( flight is full )

int Flight::generateSeat(string seatClass){ // seat randomizer
    if(seatClass == "First"){
        return rand() % 20 + 1;
    }
    else if(seatClass == "Business"){
        return rand() % 40 + 21;
    }
    else{
        return rand() % 90 + 61;
    }
}

void Flight::bookFlight(User* currentUser){
    // checking if there are available flights to be booked
    if (isFull()) {
        cout << "This flight is fully booked!\n";
        return; // stop the function
    }
    string seatClass;
    display_flight();
}
```

FLIGHT.CPP

```
cout << "Enter class (First / Business / Economy): " << endl;
cin >> seatClass;

//generate seat
int seat = generateSeat(seatClass);

//create booking
Booking b(currentUser, departure, destination, date, arrival,
           to_string(gate_number), seatClass,
           to_string(flight_num), seat); // add date and gate from booking class using composition

booking.push_back(b);
cout << "Seat assigned: " << seat << endl;
cout << "Booking successful!\n";

countBooking++; //to count num of bookings
}

string Flight::showInfo(){//showing how many bookings per flight
    string s = "Flight " + to_string(flight_num) + " to " +
               getDestination() + " has " +
               to_string(countBooking) + " bookings.";
    return s ;
}
```

MAIN.CPP

The main function gives of the effect of a flight booking website:

The user can:

- Sign-up
- Log-in
- Perform different actions (using a swtich)

The main function utilises Exception handling when taking user action to ensure the system runs neatly.